Concept Drift, Continuous ML, MLOps & CCAM: A Drift-aware Platform

Angelis Georgios, Kyriakopoulos Georgios, Tzelepis Serafeim

School of Electrical and Computer Engineering, National Technical University of Athens

28 July 2025

Agenda

1) Thesis Scope

2) Problem Statement

3) Software Design & Tools

4) Data Acquisition

5) Implementation

6) Platform Overview

7) Implementation Challenges

Thesis Scope

- Build a **drift-aware platform** for mobility models on Athens center.
- Generate **synthetic traffic data** from SUMO (Simulation of Urban MObility).
- **Three regression tasks**: ETA, Fuel Consumption, Number of Stops.
- Show end-to-end: **stable** $\rightarrow$ **drift** $\rightarrow$ **detect** $\rightarrow$ **retrain** $\rightarrow$ **mitigate**.
- Deliverables: dataset generation pipeline, tuned models for each task, platform (backend, frontend, drift service, model services).

Concept Drift & Why It Matters

- **What it is:** When the real world changes, the link between inputs and the target also changes over time (non-stationarity).
- **Why it happens:** Seasons and trends, one-off events, policy or product changes, and even sensor or software updates.
- **Why it matters:** If we don't watch for it, model accuracy quietly drops, dashboards stop matching reality, and decisions become costly or unsafe.
- **What to do:** Treat ML as a live system—**monitor** data and outcomes, **detect** changes early, and **adapt**.

Our ML Tasks

- **Estimated Time of Arrival (ETA)**: essential for accurate route planning and improving user satisfaction.
- **Fuel Consumption**: key to reducing operating costs and CO_2 emissions via eco-routing.
- **Number of Stops**: influences perceived trip quality and helps optimize overall traffic flow.

Software Design & Tools

- **Synthetic Data**: SUMO - Python.
- **Models**: XGBoost/LightGBM/CatBoost, scikit-learn, numpy, pandas - Python.
- **Backend**: FastAPI - Python.
- **Drift**: River (ADWIN, KSWIN, PageHinkley) - Python.
- **UI**: Dash/Plotly - Python.
- **Data**: Parquet/CSV files.
- **Orchestration**: Docker.

Athens Network Configurations & Traffic Patterns

- **Base network**: SUMO network built from OSM data over central Athens.
- **Rain network**: same map area, but with friction reduced in all lanes ($1.0 \rightarrow 0.4$) to simulate wet conditions.
- **Traffic generation periods**: per-hour volumes with morning and afternoon peaks, and lower mid-day activity.
- **Realistic variability**: added random noise to hourly volumes and used a distinct random seed for each 10h day simulation.

Dataset Structure

- **Three CSV files** (each ∼55–60 k trips over a 10 hour simulation):
  - `train-fcd.csv` (base network)
  - `test-fcd.csv` (base network)
  - `rain-fcd.csv` (rain-modified network)
- **Columns (per row)**: `timestep, vehicle_id, x, y, speed, lane, odometer, fuel, waiting_time`

Dataset Generation Pipeline

- **osmGet**: extract OSM data within a bounding box over central Athens
- **osmBuild**: convert OSM into a SUMO-compatible network file
- **randomTrips**: generate vehicle trips using per-hour traffic volumes and a different random seed
- **sumo**: run 10-hour simulations, export Floating Car Data (FCD) and emission logs

Data Preprocessing & Feature Engineering

- **Trip extraction**: group raw FCD by vehicle into one row per trip (origin-destination)
- **Different features** added per model:
  - *Temporal*: `hour_bin`
  - *Spatial*: source/destination clusters (`x`,`y`)
  - *Route*: number of edges traversed, distance measures (Euclidean, Manhattan, Haversine)
- **Result**: tabular dataset with inputs + ground-truth targets (ETA, fuel, stops)

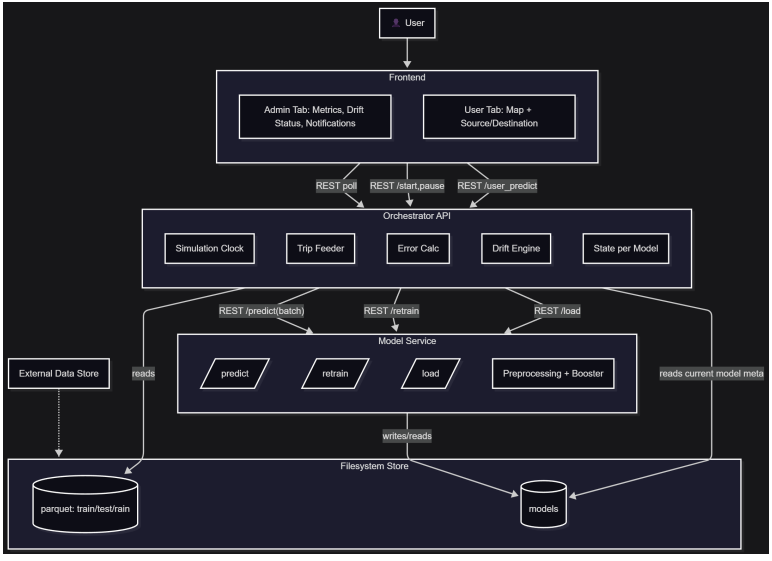
Model Training & Evaluation

- **Evaluation metrics**:
  - *Primary*: MAE
  - *Secondary*: MAPE, R^2, RMSE, MSE
- **Hyperparameter tuning**: Optuna / grid search over `n_estimators`, `max_depth`, `learning_rate`, `subsample`, ...
- **Outcome**: XGBoost delivered the best trade-off of accuracy and training speed for all three tasks

High-level Architecture

- **Frontend (Dash/Plotly)**: Admin tab (live MAE charts, notifications, run controls); User tab (Athens map, O/D query).
- **Orchestrator (FastAPI)**: simulation clock, batch prediction, error calc, drift ensemble, per-model state, UI feed, controls.
- **Model Service (FastAPI)**: per-task pipelines (preprocessing + booster), `/predict`, `/retrain`, `/load`.
- **Filesystem Store**: Parquet datasets (`train/test/rain`); filesystem model registry (`models/{task}/vN/`).
- **Docker Compose**: orchestration with multiple containers.

Components Diagram



How It Works (at a glance)

1. **Timelapse clock**: compress 20 h of trips (test then rain) into a few minutes.
2. **Per tick**: Orchestrator pulls trips whose `sim_time` falls in the current window.
3. **Batch predict**: send features to Model Service; receive predictions for ETA/Fuel/Stops.
4. **Immediate ground truth**: join with labels; compute per-trip absolute errors.
5. **Live metrics**: update rolling MAE per task and push to the UI feed.
6. **Drift monitoring**: feed a smoothed error stream into ADWIN, Page–Hinkley, KSWIN, and an SPC rule; raise drift on majority vote.
7. **Mitigation**: after drift is confirmed, collect N more "rain" trips, trigger retraining, then swap to the new model version.
8. **Repeat per model**: ETA, Fuel, Stops progress through the same state machine independently.

Admin Tab: Sped-up Simulation & Drift Story

- **Three live charts** (ETA/Fuel/Stops): rolling MAE; drift state (Stable/Confirmed/Collecting/Retraining/Swapped).
- **Notifications**: day transition at 10 h (*rain starts*), drift detected (*votes* ≥ 3), retrain started/completed, model swapped to vN.
- **What you see**: errors rise after rain; drift fires; we keep predicting with old model while "collecting" rain data; retrain runs; swap reduces error (not all the way to baseline).
- **Controls**: Start/Pause/Resume/Restart.

Admin Tab: Sped-up Simulation & Drift Story

User Tab: Predictions at the Current State

- **Map of central Athens**: user picks **source** and **destination**.
- **Uses the same simulation time**: features derive from current `sim_time` (e.g., `hour_bin`, distance, spatial bins).
- **Calls active models**: returns ETA, Fuel, Stops from whichever version is currently loaded (pre- or post-swap).
- **Consistent context**: shows badges like "Rain scenario" and current drift state so predictions match what the Admin tab shows.
- **Demo behavior**: *pauses* the timelapse while in the User tab, then *resumes* when you go back to Admin.

User Tab: Predictions at the Current State

Implementation Challenges

- **Road closures drift scenario:** Closing lanes/edges to mimic events/roadworks in SUMO led to unstable routing and distorted traffic patterns. We therefore used the rain/friction shift as a controlled, repeatable drift.
- **Parking availability task:** The current SUMO setup lacks realistic parking supply/occupancy and consistent stop events, so the task could not be integrated or evaluated alongside the trip-level models.
- **Time-series traffic speed model:** A lagged-input traffic speed model adapted quickly to the rain shift and showed little error increase, making drift hard to showcase; we replaced it with the "number of stops" task which reacts clearly.

Questions

Thank you!